Ain Shams University - Faculty of Engineering

Digital Design Verification Course - Final Project

Spring 2026

SPI Master Controller Specification

APB-slave SPI Master IP (8/16/32-bit, 4 SS, 8-deep FIFOs)

Rev 1.1 - Spring 2026

1. Overview

The SPI Master Controller is a synthesizable IP block that implements an industry-standard Serial Peripheral Interface (SPI) master with a 32-bit AMBA APB v2.0 slave interface for configuration and data access. The IP supports all four SPI modes, programmable transfer widths (8, 16, or 32 bits), up to four independent slave-select outputs, dual 8-deep FIFOs for TX and RX, a programmable clock divider, and a configurable interrupt controller. The block is designed for single-clock, synchronous, active-low-reset operation and is intended as the verification target for the Spring 2026 Digital Design Verification course final project.

1.1 Key Features

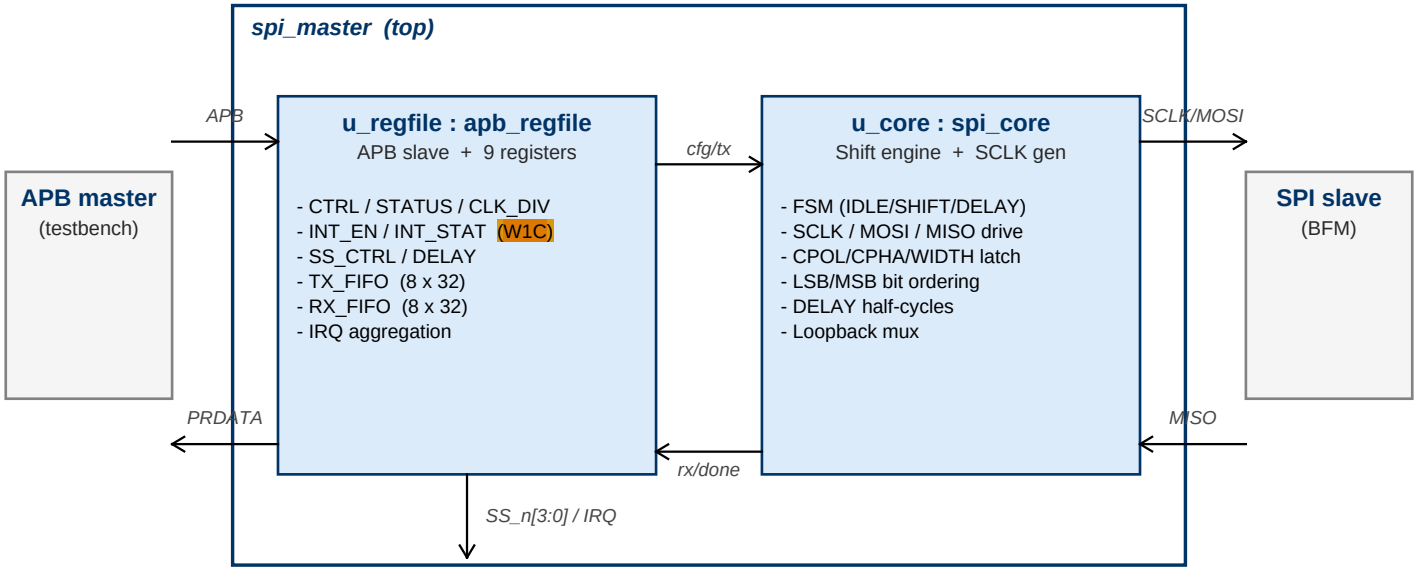
- APB v2.0 slave interface - 32-bit data, byte-addressable register file
- All four SPI modes (CPOL x CPHA)
- Programmable transfer width: 8, 16, or 32 bits
- Up to 4 independent active-low slave selects (SS_n[3:0])
- Separate 8-deep TX and RX FIFOs with full/empty and overflow status
- Programmable SCLK divider: SCLK = PCLK / (2 x (DIV+1)), DIV in [0, 65535]
- Programmable bit ordering (MSB-first or LSB-first)
- Loopback mode (MOSI driven back onto MISO internally) for self-test
- Programmable inter-transfer delay (0-255 SCLK half-cycles)
- Five maskable interrupts with sticky W1C status
- Synchronous active-low reset (PRESETn)

1.2 Pin List

Signal	Dir	Width	Description
PCLK	in	1	APB clock (also system clock for SPI logic)
PRESETn	in	1	APB active-low asynchronous reset
PSEL	in	1	APB slave select
PENABLE	in	1	APB enable phase indicator
PWRITE	in	1	APB write (1) / read (0)
PADDR	in	8	APB byte address (register offset)
PWDATA	in	32	APB write data
PRDATA	out	32	APB read data (valid when PSEL & PENABLE & PREADY & !PWRITE)
PREADY	out	1	APB ready (always 1 for zero-wait-state accesses)
PSLVERR	out	1	APB slave error (reserved - tied to 0)
SCLK	out	1	SPI serial clock
MOSI	out	1	Master-out, slave-in data
MISO	in	1	Master-in, slave-out data
SS_n[3:0]	out	4	Slave-select outputs (active low)
IRQ	out	1	Combined maskable interrupt (level, active high)

2. Architecture Overview

The IP is organized as a thin top-level module spi_master that instantiates two clearly bounded sub-blocks on the same PCLK/PRESETn domain. The top is pin-compatible with a single-file implementation; the split is purely structural and is guaranteed stable so the student verification environment may rely on the internal hierarchical names for backdoor checks and SVA bind points.



2.1 Internal Hierarchy Contract

The hierarchical names below are guaranteed by the release. The course staff will not rename these instances across any faulty-RTL variant and they may be relied on by student testbenches:

```
dut_wrapper.u_dut               : spi_master      (thin top wrapper)
dut_wrapper.u_dut.u_regfile     : apb_regfile     (APB + regs + FIFOs)
dut_wrapper.u_dut.u_core        : spi_core        (shift FSM + SCLK)
```

Typical uses: (a) backdoor reads in SV-only testbenches or UVM RAL add_hdl_path calls, and (b) bind targets for SVA that probe internal FIFO pointers or the SCLK generator without modifying RTL.

3. Programmer's Model - Register Map

All registers are 32 bits wide and accessed at 32-bit aligned byte offsets. Unused bits read as 0 and writes to them are ignored. After reset, all registers return to the reset values listed below.

Offset	Name	Access	Reset	Description
0x00	CTRL	R/W	0x0000_0000	Control register
0x04	STATUS	RO	0x0000_0012	Status register
0x08	TX_DATA	WO	0x0000_0000	TX FIFO push port
0x0C	RX_DATA	RO	0x0000_0000	RX FIFO pop port
0x10	CLK_DIV	R/W	0x0000_0000	SCLK clock divider
0x14	SS_CTRL	R/W	0x0000_0000	Slave-select enable and value
0x18	INT_EN	R/W	0x0000_0000	Interrupt enable mask
0x1C	INT_STAT	R/W1C	0x0000_0000	Interrupt status (sticky)
0x20	DELAY	R/W	0x0000_0000	Inter-transfer delay

3.1 CTRL (0x00) - Control Register

Bits	Name	Access	Description
31:8	RSVD	RO	Reserved, reads as 0
7:6	WIDTH	R/W	Transfer width: 00=8b, 01=16b, 10=32b, 11=reserved
5	LOOPBACK	R/W	1 = internally connect MOSI to MISO
4	LSB_FIRST	R/W	1 = LSB-first, 0 = MSB-first (default)
3:2	MODE	R/W	{CPOL, CPHA} - selects SPI mode 0/1/2/3
1	MSTR	R/W	Must be written 1 for master operation
0	EN	R/W	Global enable. 0 = idle, FIFOs and FSM held reset

3.2 STATUS (0x04) - Status Register (read-only)

Bits	Name	Access	Description
31:7	RSVD	RO	Reserved
6	RX_OVF	RO	RX FIFO overflow (sticky, cleared via INT_STAT)
5	TX_OVF	RO	TX FIFO overflow (sticky, cleared via INT_STAT)
4	RX_EMPTY	RO	1 = RX FIFO empty (reset value = 1)
3	RX_FULL	RO	1 = RX FIFO full (8 entries)
2	TX_EMPTY	RO	1 = TX FIFO empty
1	TX_FULL	RO	1 = TX FIFO full
0	BUSY	RO	1 = a bit-level SPI transfer is in progress

3.3 TX_DATA (0x08) - Transmit FIFO Push (write-only)

APB writes push the low WIDTH bits of PWDATA into the TX FIFO. When the FIFO is already full, the write is discarded and TX_OVF is asserted in STATUS/INT_STAT. Reads return 0 and do not pop the FIFO.

3.4 RX_DATA (0x0C) - Receive FIFO Pop (read-only)

APB reads pop one entry from the RX FIFO, zero-extended to 32 bits. When the RX FIFO is empty, reads return 0 and RX_OVF is NOT asserted (the empty read is not an error). Writes are ignored.

3.5 CLK_DIV (0x10) - Clock Divider

Bits	Name	Access	Description
31:16	RSVD	RO	Reserved
15:0	DIV	R/W	SCLK = PCLK / (2 x (DIV+1)). DIV=0 -> SCLK = PCLK/2

3.6 SS_CTRL (0x14) - Slave-Select Control

Bits	Name	Access	Description
31:8	RSVD	RO	Reserved
7:4	SS_VAL	R/W	Static drive value for SS_n when SS_EN=1 (per-bit)
3:0	SS_EN	R/W	Per-bit SS_n enable. When 1, SS_n[i]=SS_VAL[i]; when 0, SS_n[i]=1

Software is responsible for asserting SS_n (SS_EN=1, SS_VAL=0) before writing the first TX_DATA word and deasserting it after the last word has been transmitted (STATUS.BUSY = 0 and TX_EMPTY = 1).

3.7 INT_EN (0x18) and INT_STAT (0x1C)

W1C = Write 1 To Clear

Bit	Name	Event
4	TRANSFER_DONE	Asserted one PCLK cycle after each completed word shifts out
3	RX_OVF	RX FIFO overflow - word arrived while FIFO was full
2	TX_OVF	TX FIFO overflow - APB write while FIFO was full
1	RX_FULL	RX FIFO became full
0	TX_EMPTY	TX FIFO became empty

INT_STAT is sticky and cleared by writing 1 to the corresponding bit (W1C). Writing 0 has no effect. INT_EN is a simple mask: IRQ = |(INT_STAT & INT_EN). A new event setting a bit in INT_STAT takes precedence over a simultaneous W1C to the same bit (the bit remains 1 and a new edge is seen by downstream logic on the next PCLK).

3.8 DELAY (0x20) - Inter-Transfer Delay

When non-zero, the master inserts DELAY[7:0] idle SCLK half-cycles between consecutive transfers while BUSY remains 1. Writes to DELAY during an active transfer take effect starting from the next transfer.

4. SPI Protocol and Timing

4.1 SPI Modes

Mode	CPOL	CPHA	Description
0	0	0	SCLK idles low; data launched on falling edge, sampled on rising edge
1	0	1	SCLK idles low; data launched on rising edge, sampled on falling edge
2	1	0	SCLK idles high; data launched on rising edge, sampled on falling edge
3	1	1	SCLK idles high; data launched on falling edge, sampled on rising edge

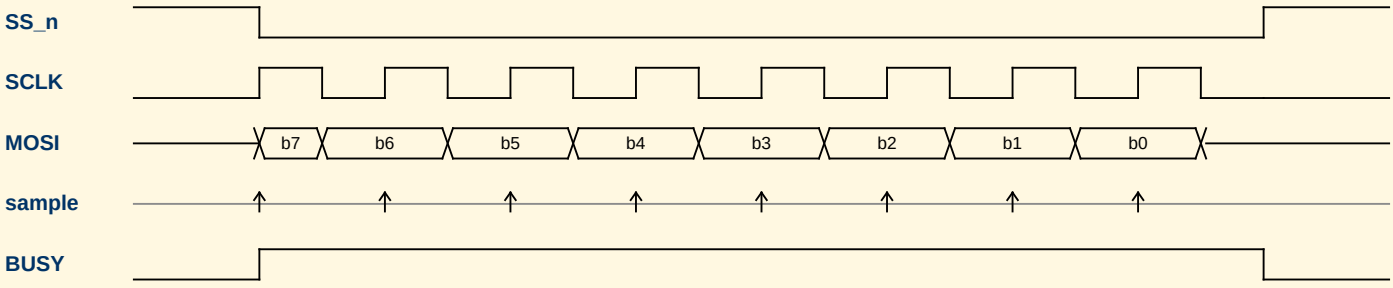
4.2 Transfer Sequence (Timing Contract)

- Precondition: CTRL.EN = 1, CTRL.MSTR = 1, at least one SS_n bit driven low via SS_CTRL, TX FIFO non-empty.
- Setup: SCLK is driven to its idle level defined by CPOL one PCLK cycle after the first valid TX word is visible and SS_n is already asserted.
- Launch / Sample edges are strictly as defined in the table above.
- Bit count per transfer: 8 / 16 / 32 per CTRL.WIDTH. Bits shift in the order defined by CTRL.LSB_FIRST.
- During the final half-SCLK cycle of the last bit, BUSY stays 1. BUSY deasserts on the PCLK edge AFTER the last sample edge.
- SS_n MUST remain asserted for the full transfer (first launch edge through BUSY deassertion). The IP never toggles SS_n automatically.
- If DELAY > 0 and another TX word is queued, the master idles on SCLK (holding idle level) for the programmed number of SCLK half-cycles before starting the next transfer. BUSY remains 1 throughout.

4.3 SCLK Timing

```
SCLK frequency:  f_SCLK = f_PCLK / (2 * (DIV+1))
SCLK duty cycle: 50% (equal high/low periods, +/- 1 PCLK)
DIV = 0   ->  SCLK = PCLK/2
DIV = 1   ->  SCLK = PCLK/4
DIV = 65535 ->  SCLK = PCLK/131072

Constraint: DIV is sampled at the start of each transfer and held
for that transfer's duration. Mid-transfer writes to CLK_DIV must
NOT alter the currently shifting transfer.
```



Illustrative waveform: 8-bit transfer in Mode 0, MSB-first, DIV=1 (SCLK period = 4 PCLK).

5. FIFOs

The TX and RX FIFOs are both 8 entries deep and 32 bits wide. Only the low WIDTH bits of each entry are used by the shifter; upper bits are don't-care on TX and zero-filled on RX.

5.1 TX FIFO

- Push: APB write to TX_DATA. Ignored silently if EN=0.
- Pop: automatic when the shift register loads the next word.
- TX_FULL / TX_EMPTY in STATUS reflect the head/tail pointers on the same PCLK edge on which they change.
- A write when TX_FULL=1 sets TX_OVF in STATUS and in INT_STAT (if unmasked, asserts IRQ). The offending write is discarded.

5.2 RX FIFO

- Push: automatic at the completion of each transfer (last sample edge).
- Pop: APB read from RX_DATA. A read when RX_EMPTY=1 returns 0 and does NOT set RX_OVF.
- A push when RX_FULL=1 sets RX_OVF; the incoming word is discarded.
- RX_FULL interrupt is asserted on the rising edge of RX_FULL.

6. APB Protocol

The register file behaves as a zero-wait-state APB v2.0 slave. PREADY is always driven 1 during ACCESS phase. PSLVERR is always 0. Reserved offsets (>= 0x24) read as 0; writes are ignored. All registers are accessed at 32-bit word granularity - narrow strobes are not supported.

```
APB access phases:
  IDLE   : PSEL=0,           -> (any write by master ignored)
  SETUP  : PSEL=1, PENABLE=0 -> slave latches PADDR, PWRITE, PWDATA
  ACCESS : PSEL=1, PENABLE=1 -> slave drives PRDATA/PREADY=1

Timing: one PCLK SETUP + one PCLK ACCESS = 2 PCLK per transaction.
```

6.1 Access-Side Effects

Register	Op	Effect
TX_DATA	Write	Pushes PWDATA[WIDTH-1:0] to TX FIFO; may set TX_OVF
TX_DATA	Read	Returns 0 (no side effect)
RX_DATA	Read	Pops one RX FIFO entry; returns 0 if empty
RX_DATA	Write	Ignored
INT_STAT	Write	W1C: bits written as 1 are cleared; 0 has no effect
CTRL.EN 1->0	Write	Flushes TX/RX FIFOs and resets shifter; does not clear INT_STAT

7. Reset, Clocking, and Interrupts

7.1 Reset

- PRESETn is active-low asynchronous assertion, synchronous deassertion (assumed by the IP's internal reset synchronizer - external sync is not required).
- On reset: all R/W registers return to their reset values; TX and RX FIFOs are emptied; shifter is cleared; SCLK is driven to the CPOL=0 idle (low) level until CTRL is programmed.
- Minimum assertion: PRESETn must be held low for at least 2 PCLK cycles.

7.2 Clocking

- Single clock domain: PCLK clocks both the APB side and the SPI shifter.
- SCLK is a gated, divided version of PCLK - it is NOT a separate clock for timing-closure purposes; all flip-flops are clocked by PCLK.
- Maximum PCLK frequency is implementation-defined (target >= 100 MHz).

7.3 Interrupts

The IP has one combined, level-sensitive, active-high interrupt output IRQ. The following equation defines IRQ at all times:

```
  IRQ = | (INT_STAT & INT_EN)
```

- INT_STAT is sticky: once a bit is set by its event, it remains 1 until explicitly cleared by a W1C from software.
- Write-1-to-clear race: if the event source asserts on the same PCLK cycle that software writes 1 to that bit, the bit STAYS 1 and the new event is captured. No events are lost.
- Masked events still set INT_STAT; INT_EN only gates IRQ, not the sticky status latches.
- After PRESETn, all INT_STAT bits are 0 and IRQ is 0.

8. Programmer's Sequences

8.1 Basic 8-bit Transfer (Mode 0, MSB-first, SS_n[0])

```
apb_write(CLK_DIV , 32'h0000_0003);        // SCLK = PCLK/8
apb_write(CTRL    , 32'h0000_0003);        // EN=1, MSTR=1, Mode 0, 8-bit
apb_write(SS_CTRL , 32'h0000_0001);        // SS_EN[0]=1, SS_VAL[0]=0
apb_write(TX_DATA , 32'h0000_00A5);        // Push 0xA5
// poll until BUSY=0 AND TX_EMPTY=1
do { s = apb_read(STATUS); } while (s[0] || !s[2]);
apb_write(SS_CTRL , 32'h0000_0000);        // Deassert SS_n
rx = apb_read(RX_DATA);                    // Received byte
```

8.2 Burst of 4 16-bit Words, Interrupt-Driven

```
apb_write(CTRL   , 32'h0000_0043);         // EN=1, MSTR=1, WIDTH=16b
apb_write(INT_EN , 32'h0000_0011);         // TX_EMPTY + TRANSFER_DONE
apb_write(SS_CTRL, 32'h0000_0001);
for (i=0; i<4; i++) apb_write(TX_DATA, data[i]);
// ISR clears INT_STAT bits via W1C and drains RX_DATA
```

8.3 Illegal Sequences (undefined behavior)

- Writing CTRL.WIDTH = 2'b11.
- Writing TX_DATA while CTRL.EN = 0 (write is simply ignored, but the software bug of enqueuing data before enabling the block is not masked).
- Changing CTRL.MODE, CTRL.WIDTH, CTRL.LSB_FIRST, or CLK_DIV while STATUS.BUSY = 1.
- Byte or halfword APB accesses (non-32-bit).

9. Numbered Functional Requirements

These requirements are the authoritative contract against which the design is verified. Every requirement must be covered by at least one test and one functional coverage bin or assertion.

ID	Requirement
R1	APB reads and writes to all R/W registers return exactly the last written value (masked by the RO bits).
R2	All registers return their specified reset values after PRESETn asserts.
R3	CTRL.EN=0 holds the shifter and FIFOs in reset; SCLK stays at CPOL idle; SS_n forced high regardless of SS_CTRL.
R4	For each SPI mode, SCLK idle polarity matches CPOL before, between, and after transfers.
R5	For each SPI mode, MOSI is stable across the sample edge defined by CPOL/CPHA and changes on the launch edge.
R6	MSB-first shifts bit [WIDTH-1] first; LSB-first shifts bit [0] first (both on TX and RX).
R7	A transfer lasts exactly WIDTH SCLK cycles; BUSY=1 throughout and deasserts one PCLK after the last sample edge.
R8	SCLK frequency equals PCLK / (2 x (DIV+1)) for all DIV in [0, 65535].
R9	TX_DATA writes are accepted while !TX_FULL, pushed in FIFO order.
R10	RX_DATA reads pop RX FIFO in FIFO order when !RX_EMPTY.
R11	TX FIFO depth is exactly 8 entries; TX_FULL asserts on the 8th pending entry.
R12	RX FIFO depth is exactly 8 entries; RX_FULL asserts on the 8th received entry.
R13	A TX_DATA write while TX_FULL=1 discards the write and sets STATUS.TX_OVF and INT_STAT[TX_OVF].
R14	A transfer completing while RX_FULL=1 discards the received word and sets STATUS.RX_OVF and INT_STAT[RX_OVF].
R15	RX_DATA read while RX_EMPTY returns 0 and does NOT set RX_OVF.
R16	IRQ = |(INT_STAT & INT_EN) at all times; INT_EN does not gate status capture.
R17	INT_STAT is W1C: writing 1 to a bit clears it; 0 has no effect.
R18	W1C race: if an event sets a bit on the same PCLK as a W1C of that bit, the bit remains 1.
R19	Loopback mode (CTRL.LOOPBACK=1) routes MOSI internally to the RX shift register; external MISO is ignored.
R20	SS_n[i] = !SS_EN[i] | SS_VAL[i] combinationally; IP never drives SS_n autonomously.
R21	DELAY SCLK half-cycles of idle are inserted between consecutive transfers when DELAY > 0 and another word is queued.
R22	APB PSLVERR is 0 and PREADY is 1 for every addressed access (zero wait states).
R23	Reserved offsets (0x24+) read as 0 and writes are ignored.
R24	CLK_DIV=0 yields SCLK = PCLK/2 (DIV is not a divide-by-zero).
R25	DIV, MODE, WIDTH, LSB_FIRST are sampled at transfer start and held for that transfer.

10. Verification Targets

This section is informative and lists the minimum targets the student testbench must achieve. It is NOT part of the RTL contract.

10.1 Functional Coverage (minimum 85% on golden RTL)

- All 4 SPI modes x all 3 widths = 12 combinations, each with MSB-first and LSB-first -> 24 bins.
- CLK_DIV corners: 0, 1, 2, 3, 255, 1024, 65535, plus a random covering bin over the full range.
- FIFO occupancy bins: empty, 1, mid (4), 7, full for both TX and RX.
- Each of the 5 interrupts: asserted, cleared via W1C, asserted while masked (status set, IRQ still 0).
- Each register: written, read back, reset-value observed.
- DELAY bins: 0, 1, and one large value (>= 128).
- Loopback mode: at least one transfer per width.

10.2 Mandatory Assertions (SVA)

- APB: PSEL=1 for at least 2 PCLK to complete a transaction.
- APB: PENABLE must only assert while PSEL=1.
- APB: PADDR, PWRITE, PWDATA stable from SETUP to ACCESS of the same transaction.
- SPI: SCLK idle level matches CPOL whenever BUSY=0.
- SPI: MOSI stable for at least 1 PCLK around each sample edge (WIRE-STABILITY).
- SPI: SS_n held asserted for the entire WIDTH-bit transfer.
- FIFO: no push when full (after OVF clear) without explicit OVF assertion.
- IRQ: IRQ == |(INT_STAT & INT_EN) every PCLK (combinational assertion).

10.3 Regression Budget

The student regression must close all coverage goals and detect all grader-injected bugs in no more than 10,000 simulation invocations total across the full test list.

11. Glossary and Revision History

Term	Definition
APB	AMBA Peripheral Bus (ARM bus standard)
BUSY	STATUS[0] - 1 while a bit-level transfer is active
CPHA	SPI Clock Phase - selects which SCLK edge is the sample edge
CPOL	SPI Clock Polarity - selects the SCLK idle level
DIV	CLK_DIV[15:0] - SCLK divider value
FIFO	First-In-First-Out queue (8 entries, 32 bits wide here)
IRQ	Combined level-sensitive interrupt output
MISO	Master-In Slave-Out serial data
MOSI	Master-Out Slave-In serial data
PCLK	APB clock (also the only clock for SPI logic)
SCLK	SPI serial clock generated by the master
SS_n	Active-low slave select
SPI mode	Concatenation {CPOL, CPHA} - one of 0/1/2/3
W1C	Write-1-to-Clear access type
WIDTH	CTRL[7:6] - transfer width (8/16/32 bits)

Revision History

Revision	Date	Notes
1.0	Spring 2026	Initial release for DDV course final project
1.1	Spring 2026	Added Section 2 (architecture block diagram + exposed internal hierarchy contract for apb_regfile / spi_core sub-blocks)

End of Specification

This specification is the sole source of truth for the SPI Master IP behavior. In the event of a conflict between this document and the released golden RTL, the specification wins and a clarification memo will be issued via the course announcements channel.